FIFO

What is FIFO?

FIFO is temporary memory element. Where there is mismatch in data coming in and going out.

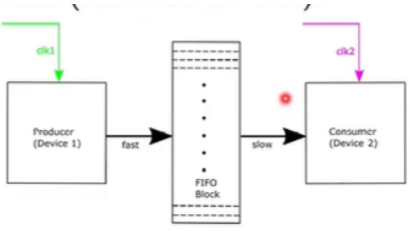
Types of FIFO

Their is two types of FIFO.

- What is Asynchronous FIFO

  In this we use 2 clocks.

  - Read and write clocks different
  - Used for Clock Domain Crossing (CDC)
  - Uses:
    - Gray code pointers
    - 2-stage synchronizers

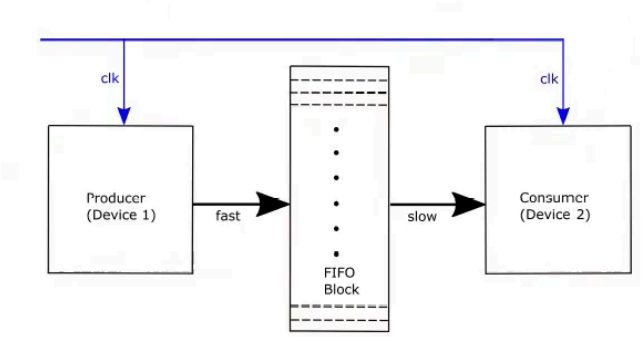


- What is Synchronous FIFO

  We use only one clock for read and write operation

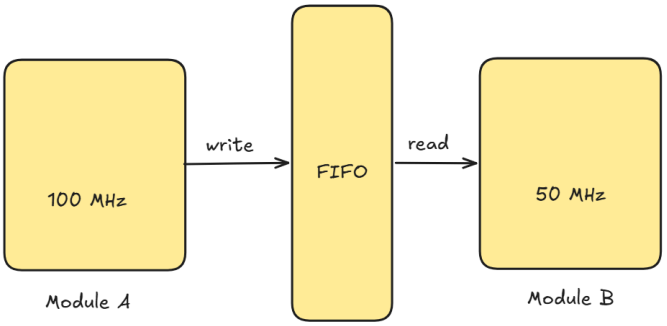
  - Same clock for read and write
  - Simple pointer comparison

- Used inside single clock domain blocks

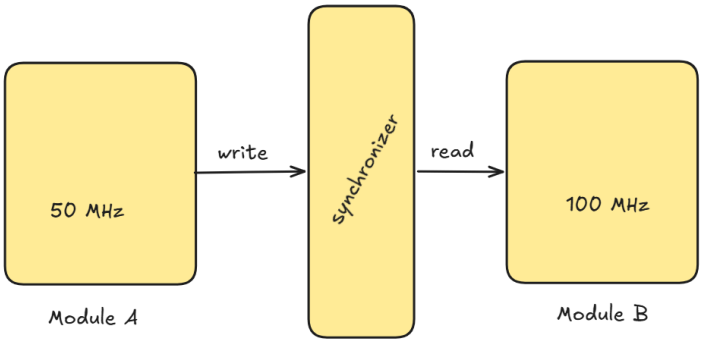


When to use FIFO

Case 1 - when the write speed > read speed. then use fifo to store excess data

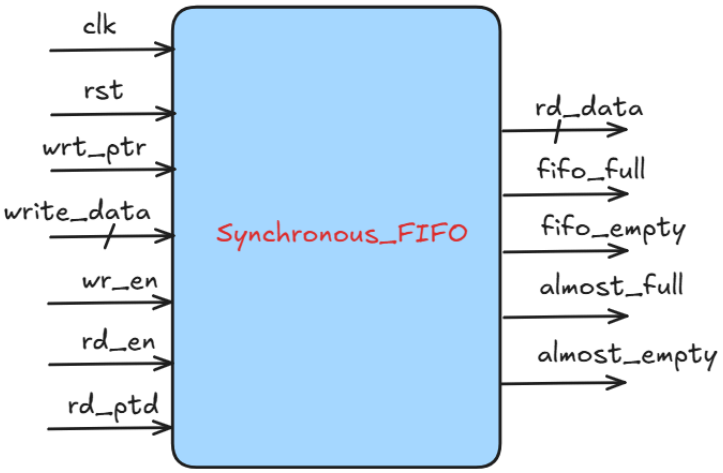


Case 2 - Read speed > Write speed. That time we didn't required fifo. we have to use Synchronizer



💡 **In fifo,**
Data transmission is happens in Burst mode not in continuous bit by bit mode.
Burst - Data packets

Synchronous FIFO



💡

- **almost_full : to avoid Overrun**
- **almost_empty : to avoid Underrun**

Design in Verilog

```
module sync_fifo #(
    parameter WIDTH = 32,
    parameter DEPTH = 16
)(
    input wire clk,
    input wire rst,

    input wire wr_en,
    input wire rd_en,

    input wire [WIDTH-1:0] data_in,
    output reg [WIDTH-1:0] data_out,

    output wire full,
    output wire empty,
    output wire almost_full,
    output wire almost_empty
);

    // memory
    reg [WIDTH-1:0] mem [0:DEPTH-1];
```

```
    // pointers
    reg [$clog2(DEPTH)-1:0] wr_ptr;
    reg [$clog2(DEPTH)-1:0] rd_ptr;

    // fifo counter
    reg [$clog2(DEPTH):0] fifo_count;


    // WRITE LOGIC
    always @(posedge clk) begin
        if (rst)
            wr_ptr <= 0;

        else if (wr_en && !full) begin
            mem[wr_ptr] <= data_in;
            wr_ptr <= wr_ptr + 1;
        end
    end


    // READ LOGIC
    always @(posedge clk) begin
        if (rst)
            rd_ptr <= 0;

        else if (rd_en && !empty) begin
            data_out <= mem[rd_ptr];
            rd_ptr <= rd_ptr + 1;
        end
    end


    // FIFO COUNTER UPDATE
    always @(posedge clk) begin
        if (rst)
            fifo_count <= 0;

        else begin
            case ({wr_en && !full, rd_en && !empty})

                2'b10: fifo_count <= fifo_count + 1; // write only
                2'b01: fifo_count <= fifo_count - 1; // read only
                2'b11: fifo_count <= fifo_count;     // read + write same
cycle
                default: fifo_count <= fifo_count;

            endcase
```

```
        end
    end


    // STATUS FLAGS
    assign full  = (fifo_count == DEPTH);
    assign empty = (fifo_count == 0);

    assign almost_full  = (fifo_count >= DEPTH-2);
    assign almost_empty = (fifo_count <= 1);

endmodule
```

Verification by Using Cocotb

```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge


async def reset(dut):
    dut.rst.value = 1
    dut.wr_en.value = 0
    dut.rd_en.value = 0
    dut.data_in.value = 0

    await RisingEdge(dut.clk)
    await RisingEdge(dut.clk)

    dut.rst.value = 0
    await RisingEdge(dut.clk)


@cocotb.test()
async def fifo_test(dut):

    cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())

    await reset(dut)

    # ----------------
    # WRITE PHASE
    # ----------------

    for i in range(4):

        dut.wr_en.value = 1
        dut.data_in.value = i
```

```
        await RisingEdge(dut.clk)

    dut.wr_en.value = 0

    # allow fifo to settle
    await RisingEdge(dut.clk)

    # ----------------
    # READ PHASE
    # ----------------

    for expected in range(4):

        dut.rd_en.value = 1
        await RisingEdge(dut.clk)

        dut.rd_en.value = 0
        await RisingEdge(dut.clk)

        data = dut.data_out.value.to_unsigned()

        dut._log.info(f"READ = {data}")

        assert data == expected, f"Expected {expected}, got {data}"
```